Laboratory Activity 3: Modular Array Operations & Bounds-Checked Roster Management
NTC CC105 – Data Structure w/Lab
School of Arts, Sciences, and Technology | National Teachers College
Academic Year 2026–2027 | First Semester

```
#include <iostream>
#include <string>
#include <iomanip>

using namespace std;

const int MAX_CAPACITY = 50;

struct Student {
    int id;
    string name;
};

// Function Prototypes
int findStudentById(const Student roster[], int count, int targetId, int &comparisons);
bool addStudent(Student roster[], int &count, int id, const string &name);
bool removeStudentById(Student roster[], int &count, int targetId);
bool safeGetRecord(const Student roster[], int count, int index, Student &outStudent);
void printRoster(const Student roster[], int count);

int main() {
    Student roster[MAX_CAPACITY];
    int currentCount = 0;
    int choice = 0;

    do {
        cout << "\n========================================\n";
        cout << " NTC CC105: STUDENT ROSTER MANAGER \n";
        cout << "========================================\n";
        cout << "1. Add Student Record\n";
        cout << "2. Search Student by ID\n";
        cout << "3. Remove Student by ID\n";
        cout << "4. Safe Inspect Record by Index\n";
        cout << "5. Display Full Roster\n";
        cout << "6. Exit\n";
        cout << "Enter selection [1-6]: ";
        cin >> choice;
```

```
switch (choice) {
    case 1: {
        if (currentCount >= MAX_CAPACITY) {
            cout << "Error: Full na ang roster!\n";
            break;
        }

        int id;
        string name;

        cout << "Enter Student ID: ";
        cin >> id;

        cin.ignore(); // Linisin ang buffer bago mag-getline
        cout << "Enter Student Name: ";
        getline(cin, name);

        if (addStudent(roster, currentCount, id, name)) {
            cout << "Success: Naidagdag na ang estudyante!\n";
        }
        break;
    }
    case 2: {
        if (currentCount == 0) {
            cout << "Warning: Walang laman ang roster.\n";
            break;
        }

        int targetId;
        int comparisons = 0;

        cout << "Enter Student ID to search: ";
        cin >> targetId;

        int index = findStudentById(roster, currentCount, targetId, comparisons);
        if (index != -1) {
            cout << "Record Found at Index [" << index << "]:\n";
            cout << "   ID   : " << roster[index].id << "\n";
            cout << "   Name : " << roster[index].name << "\n";
        } else {
            cout << "Record Not Found: Walang ID na " << targetId << ".\n";
        }
        cout << "Total comparisons: " << comparisons << "\n";
```

```
    break;
}
case 3: {
    if (currentCount == 0) {
        cout << "Warning: Walang laman ang roster.\n";
        break;
    }

    int targetId;
    cout << "Enter Student ID to remove: ";
    cin >> targetId;

    if (removeStudentById(roster, currentCount, targetId)) {
        cout << "Success: Natanggal ang estudyante at na-shift ang array.\n";
    } else {
        cout << "Error: Hindi nahanap ang ID na " << targetId << ".\n";
    }
    break;
}
case 4: {
    if (currentCount == 0) {
        cout << "Warning: Walang laman ang roster.\n";
        break;
    }

    int index;
    cout << "Enter index to inspect (0 to " << currentCount - 1 << "): ";
    cin >> index;

    Student retrieved;
    if (safeGetRecord(roster, currentCount, index, retrieved)) {
        cout << "Success: Record at index [" << index << "]:\n";
        cout << "   ID   : " << retrieved.id << "\n";
        cout << "   Name : " << retrieved.name << "\n";
    } else {
        cout << "Error: Invalid index [" << index << "]! Out of bounds.\n";
    }
    break;
}
case 5: {
    printRoster(roster, currentCount);
    break;
}
case 6: {
```

```
            cout << "Exiting system. Memory cleaned successfully.\n";
            break;
        }
        default: {
            cout << "Invalid selection! Pumili lang mula 1 hanggang 6.\n";
            break;
        }
    }
  } while (choice != 6);

  return 0;
}

//
================================================================================
====
// FUNCTIONS IMPLEMENTATION
//
================================================================================
====

// 1. Search ID gamit ang Linear Search
int findStudentById(const Student roster[], int count, int targetId, int &comparisons) {
  comparisons = 0;
  for (int i = 0; i < count; i++) {
    comparisons++;
    if (roster[i].id == targetId) {
      return i;
    }
  }
  return -1;
}

// 2. Add Student (Sinisigurong Unique ang ID)
bool addStudent(Student roster[], int &count, int id, const string &name) {
  if (count >= MAX_CAPACITY) {
    return false;
  }

  int dummyComp = 0;
  if (findStudentById(roster, count, id, dummyComp) != -1) {
    cout << "Error: May kaparehong ID na sa roster!\n";
    return false;
  }
```

```
    roster[count].id = id;
    roster[count].name = name;
    count++;
    return true;
}

// 3. Remove Student (Left-shifting ng elements)
bool removeStudentById(Student roster[], int &count, int targetId) {
    int dummyComp = 0;
    int targetIndex = findStudentById(roster, count, targetId, dummyComp);

    if (targetIndex == -1) {
        return false;
    }

    // Left-shift para isara ang bakanteng espasyo
    for (int i = targetIndex; i < count - 1; i++) {
        roster[i] = roster[i + 1];
    }

    count--;
    return true;
}

// 4. Safe Get Record (Bounds Checking)
bool safeGetRecord(const Student roster[], int count, int index, Student &outStudent) {
    if (index < 0 || index >= count) {
        return false;
    }
    outStudent = roster[index];
    return true;
}

// 5. Display Full Roster
void printRoster(const Student roster[], int count) {
    if (count == 0) {
        cout << "Warning: Walang laman ang roster.\n";
        return;
    }

    cout << "\n----------------------------------------\n";
    cout << left << setw(8) << "Index" << setw(12) << "ID" << "Name\n";
    cout << "----------------------------------------\n";
```

```
    for (int i = 0; i < count; i++) {
        cout << left << setw(8) << i
             << setw(12) << roster[i].id
             << roster[i].name << "\n";
    }

    cout << "----------------------------------------\n";
    cout << "Total Records: " << count << " / " << MAX_CAPACITY << "\n";
}
```

```
5. Display Full Roster
6. Exit
Enter selection [1-6]: 1
Enter Student ID: 2
Enter Student Name: 3
Success: Naidagdag na ang estudyante!

========================================
 NTC CC105: STUDENT ROSTER MANAGER
========================================
1. Add Student Record
2. Search Student by ID
3. Remove Student by ID
4. Safe Inspect Record by Index
5. Display Full Roster
6. Exit
Enter selection [1-6]: 4
Enter index to inspect (0 to 0): 5
Error: Invalid index [5]! Out of bounds.

========================================
 NTC CC105: STUDENT ROSTER MANAGER
========================================
1. Add Student Record
2. Search Student by ID
3. Remove Student by ID
4. Safe Inspect Record by Index
5. Display Full Roster
6. Exit
Enter selection [1-6]:
```

```
================================================================================
5. PERFORMANCE REFLECTION (WRITTEN TASK)
================================================================================

Question 1: Random Access vs. Linear Search
Why does inspecting a record via safeGetRecord execute in O(1) time complexity,
whereas searching for an ID via findStudentById requires O(n) worst-case complexity?

Answer:
`safeGetRecord` uses O(1) constant time complexity because array elements are stored
contiguously in memory. The memory address is calculated directly via formula:
    Address = Base Address + (Index * Element Size)
This formula runs in 1 exact step regardless of array size.

`findStudentById` uses Linear Search with O(n) worst-case complexity because it checks
elements sequentially starting from index 0. If the target is at the end or missing,
it must perform `n` comparisons.


--------------------------------------------------------------------------------

Question 2: Deletion Cost
Explain why deleting an element from index 0 is more computationally expensive
than deleting an element from index currentCount - 1.

Answer:
Deleting at index 0 requires shifting all remaining `n - 1` elements left by one position
to maintain contiguous memory, requiring O(n) steps.

Deleting at index `currentCount - 1` (the tail) requires no shifting. The system simply
decrements `count` by 1, completing in O(1) constant time.
================================================================================
*/
```